Лекция 2. CV-квалификаторы. Ссылки. Указатели

1. CV-квалификаторы
   - `const`
   - `volatile`
   - `const volatile`
   - Оператор `const_cast`
2. Ссылка (*reference*)
   - Оператор доступа `[]`
   - Ссылка на константу
   - Ссылка на массив
   - Итерирование по контейнеру
3. Указатель (*pointer*)
   - Оператор взятия адреса `&` (*address-of*)
   - Использование амперсанда `&`
   - Получение адреса `std::addressof()` (*C++11*)
   - Оператор разыменования `*` (*indirection*)
   - Использование звездочки `*`
   - Стиль кода амперсанда и звездочки
   - Совместное использование амперсанда и звездочки
   - Константность указателя
   - Нулевой указатель `nullptr`
   - Арифметика указателей
   - Сравнение указателей
   - Расстояние между указателями
   - Связь массивов и указателей
   - Передача массива через указатели
   - Ссылка на указатель
   - Указатель на указатель
   - Указатель на `void`
   - Оператор `reinterpret_cast`
   - Правило строгого алиасинга (*strict aliasing*)
   - Указатель на функцию
   - Преобразование *function to function pointer*
   - Сложные объявления с указателями
4. Передача аргументов в функцию
   - Продление жизни временного объекта ссылкой на константу
   - Почему нет продления жизни неконстантной ссылки
   - Висячая ссылка (*dangling reference*)
   - Невалидный указатель (*invalid pointer*)

CV-квалификаторы

В языке C++ существуют квалификаторы типа.

CV означает `const` `volatile`

Квалификаторы могут быть применены к типам переменных и методам (функциям внутри класса)

`const`

Квалификатор типа, отвечающий за константность.

Означает, что объект данного типа нельзя изменять

Если тип объявлен с данным квалификатором, то компилятор не разрешает пользователю изменять данную сущность. Что позволяет находить ошибки в коде на этапе компиляции (например, при попытке изменить переменную с квалификатором `const`)

Все глобальные переменные рекомендуется объявлять константными.

Можно написать как **ДО** типа, так и **ПОСЛЕ**:

```
const int a = 0;
int const b = 0;
```

- **НО** принято писать **ДО** типа

Невозможно делать операции с переменной данного типа, подразумевающие изменение (инкремент/декремент, присваивания, ...)

`volatile`

Квалификатор типа, отвечающий за изменчивость.

Означает, что значение объекта может измениться вне программы, неожиданно для компилятора, а запись в объект может иметь побочный эффект вне программы.

Сообщает компилятору, что переменная видима извне, а значит **НЕ** нужно оптимизировать код, связанный с данной переменной. Каждое чтение и каждая запись `volatile` объекта должны выполняться на самом деле, в том порядке и количестве, как написано в коде. Компилятор не вправе их выбрасывать, объединять или кэшировать значение в регистре процессора.

```
volatile int a = 0;
int volatile b = 0;
```

Обычно компилятор оптимизирует работу с памятью. Если компилятор видит, что переменную никто не меняет, он прочитает её один раз. Если значение записывается дважды подряд, он оставит только последнюю запись.

Для обычных переменных это корректно, но **НЕ** для **регистров устройств**.

В микроконтроллерах периферия (порты ввода-вывода, UART, таймеры, АЦП) управляется через ячейки памяти по фиксированным адресам согласно документации на микросхему. Такой способ называется *memory-mapped I/O* (отображение ввода-вывода на память): запись по адресу отправляет команду устройству, чтение по другому адресу — получает его состояние.

В коде регистр выглядит как обычный указатель на заданный адрес:

```
// адреса и биты - из документации на микроконтроллер
volatile std::uint32_t* const UART_STATUS =
    reinterpret_cast<volatile std::uint32_t*>(0x40011000);
volatile std::uint32_t* const UART_DATA =
    reinterpret_cast<volatile std::uint32_t*>(0x40011004);

constexpr std::uint32_t RX_READY = 1u << 5;

// ждать, пока устройство не получит байт
while ((*UART_STATUS & RX_READY) == 0) {
    // некоторый код
}
std::uint32_t byte = *UART_DATA;
```

- `reinterpret_cast` нужен, потому что целое число нельзя неявно преобразовать в указатель

Значение `*UART_STATUS` меняет **устройство**, а не программа. Без `volatile` компилятор видит цикл, в котором память никто **НЕ** изменяет, и считает, что результат проверки всегда одинаков и известен на этапе компиляции.

Без `volatile` компилятор при оптимизации полностью выкинет цикл из ассемблерного кода

При записи двух разных значений, идущих подряд, в обычную переменную компилятор оставляет только последнюю, а для `volatile` переменной будут выполнены обе. Для регистра устройства это может иметь особое значение: например, `1`, а затем `0` — это импульс сброса.

```
void WriteTwice(volatile std::uint32_t* reg) {
    *reg = 1; // без volatile эта запись была бы удалена
    *reg = 0;
}
```

Частая ошибка — использовать `volatile` для переменных, которые читаются и изменяются из разных потоков. Квалификатор `volatile` **НЕ** предназначен для синхронизации в многопоточности:

- Операции с `volatile` переменной **НЕ** атомарны: увеличение счетчика из двух потоков теряет инкременты.
- Квалификатор `volatile` **НЕ** упорядочивает доступ к другим переменным и **НЕ** синхронизирует кэши процессорных ядер.
- Одновременный доступ к переменной из разных потоков без синхронизации (даже `volatile`) — это *data race*, то есть **UB**.

Для многопоточности используются `std::atomic`, мьютексы и другие средства синхронизации.

Использование квалификатора `volatile`:

- Регистры устройств и *memory-mapped I/O* во встраиваемых системах.
- Переменные, изменяемые в обработчике сигнала операционной системы.
- Бенчмарки, чтобы компилятор **НЕ** выбросил «бесполезное» вычисление.

В обычном прикладном коде `volatile` практически **НЕ** встречается.

`const volatile`

Можно использовать оба квалификатора.

Означает, что объект нельзя изменять пользователю, но он может неожиданно измениться вне программы.

Оператор `const_cast`

`const_cast` - оператор приведения типа, который необходим для добавления или удаления квалификаторов `const`, `volatile`.

Синтаксис: `const_cast<new_type>(<expression>)`

- `new_type` - тип, к которому необходимо привести
- `<expression>` - выражение, результат которого необходимо привести к указанному типу

Оператор `const_cast` используется:

- В *legacy* коде для старых C-API, когда принимается **НЕ** `const` указатель, хотя данные не изменяются.
- Для удаления `const` с объектов, которые не были объявлены `const`, но стали константными в результате передачи в функцию.

ВАЖНО. Изменение через `const_cast` объекта, который изначально объявлен `const` — это **UB**.

```
void change_const_value(const int& value) {
    int& mutable_value = const_cast<int&>(value);
```

```
    mutable_value = 100; // только если оригинал не const
}

int main() {
    int original = 42;
    std::cout << "before const_cast: " << original << std::endl;
    change_const_value(original);
    std::cout << "after const_cast: " << original << std::endl;
}
```

Без особой необходимости **НЕ** следует использовать `const_cast`.

Ссылка (*reference*)

Ссылка представляет собой псевдоним для переменной.

Ссылка позволяет использовать объект, избегая копирования, ссылка **НЕ** хранит значение.

Для создания ссылки используется амперсанд `&` после имени типа.

Синтаксис: `<type>& <alias_name> = <exist_name>;`

- `<alias_name>` - имя ссылки
- `<exist_name>` - имя существующей переменной

Представляет собой новый составной тип: ссылка на тип.

Особенности:

- нельзя создать ссылку без инициализации
- ссылка может быть связана с переменной один раз
- ссылки на ссылку не существует

Ссылка позволяет изменять исходный объект: `<ref_name> = <value>`

```
int x = 0;
int& y = x;
y = 1;
std::cout << x; // 1
```

```
int x = 0, z = 5;
int& y = x;
y = z;          // change x, not change ref for y
std::cout << x; // 5
```

Оператор доступа `[]`

Оператор `[]` возвращает ссылку на объект:

```
int arr[] = {1, 2, 3, 4, 5};
int& value = arr[2];
value = 0;  // arr = {1, 2, 0, 4, 5}
arr[3] = 8; // arr = {1, 2, 0, 8, 5}
```

Ссылка на константу

Ссылку на константу часто называют константной ссылкой.

При добавлении константности к ссылке **нельзя** поменять переменную через ссылку.

```
int x = 0;
const int& ref = x;
ref = 1; // compile error
```

На константную переменную может ссылаться только ссылка на константу:

```
const int value = 100;
int& ref = value;        // compile error
const int& cref = value; // ok
```

На литерал нельзя сделать обычную ссылку, потому что литерал — это константа времени компиляции, *rvalue* (временное значение без адреса). Но можно сделать константную ссылку на литерал, поскольку в языке принято продление жизни константных ссылок (*lifetime extension*)

```
int& ref = 10;        // compile error
const int& cref = 10; // ok
```

- В этот момент создается объект со значением литерала, на который создается ссылка

Ссылка на массив

Ссылку на массив можно создать следующим образом:

```
int arr[10];
int (&ref_a)[10] = arr;
int matrix[3][4];
int (&ref_m)[3][4] = matrix;
```

- Придется явно указать размерность.

- При передаче в функцию сохраняется возможность итерироваться по массиву в цикле *range-based for*

Итерирование по контейнеру

Для тяжелых типов важно правильно итерироваться по контейнерам и стандартным массивам.

Спецификатор типа `auto` срезает тип, а именно ссылку и константность. Поэтому его нужно уточнять с помощью `const` и `&`.

Копирование не подходит для тяжелых объектов (`std::string` - строка C++, может быть тяжелым объектом).

```
std::vector<std::string> book_names;
// some code to fill book_names...
for (auto book_name : book_names) {
    // do something...
}
```

Если необходимо изменять объект в цикле, то используется ссылка `auto&`. Для тяжелых объектов позволяет избежать копирования и значительно ускорить итерирование по контейнеру.

```
std::vector<std::string> book_names;
// some code to fill book_names...
for (auto& book_name : book_names) {
    // change book_names strings through ref book_name...
}
```

Если **НЕТ** необходимости менять объект, используется `const auto&`

```
std::vector<std::string> book_names;
// some code to fill book_names...
for (const auto& book_name : book_names) {
    // do something without changing of book_name...
}
```

ВАЖНО для легких типов (фундаментальных, перечислений, типов до 8-16 байт), если нет необходимости изменять объект, лучше использовать не `const auto&`, а копирование через `auto`. Это будет быстрее.

```
std::vector<int> values;
// some code to fill values...
for (auto value : values) {
    std::cout << value << std::endl;
}
```

Указатель (*pointer*)

Указатель — это тип, который позволяет хранить адрес другого объекта в памяти.

Синтаксис: `<type>* <name> = <address>;`

- `*` - обозначение указателя
- `<type>` - тип объекта, на который указывает указатель
- `<name>` - имя для указателя, иногда начинается с `ptr_`
- `<address>` - адрес объекта в памяти, на который указывает указатель

```
unsigned x = 0xBADC0FFE;
unsigned* ptr_x = &x;
std::cout << x << std::endl;
std::cout << ptr_x << std::endl;
std::cout << *ptr_x << std::endl;
```

Особенности:

- указатели могут быть не инициализированы (но так не стоит делать)
- можно перенаправить указатель на адрес другого объекта
- можно создавать указатель на указатель

Оператор взятия адреса `&` (*address-of*)

- Унарный оператор, который используется перед именем сущности для взятия адреса.

```
int x = 42;
std::cout << &x << std::endl;  // address-of x
const double pi = 3.1415926;
std::cout << &pi << std::endl; // address-of pi
int& y = x;
std::cout << &y << std::endl;  // address-of x
```

Использование амперсанда `&`

- Символ используется в разных контекстах:

```
unsigned x = 0;
// Побитовое И
auto y = x & 0xBADC0FFE;
// Объявление ссылки
unsigned& ref = x;
```

```
// Взятие адреса
auto addr = &x;
```

Получение адреса `std::addressof()` (*C++11*)

В стандартной библиотеке языка C++ в заголовочном файле `<memory>` присутствует шаблонная функция `std::addressof`, которая позволяет взять правильный адрес объекта, даже если перегружен оператор амперсанд `&`.

Функция возвращает указатель на тип аргумента (адрес аргумента).

```
int x = 10;
std::cout << std::addressof(x) << std::endl;
```

Оператор разыменования `*` (*indirection*)

- Унарный оператор, позволяющий по адресу получить доступ к объекту, который лежит по данному адресу. Называется `indirection` или `dereference`.

```
unsigned x = 0xBADC0FFE;
unsigned* ptr_x = &x;
std::cout << *ptr_x << std::endl; // dereference ptr_x
unsigned y = *ptr_x; // dereference ptr_x
```

Как правило, возвращает ссылку на объект, поэтому можно изменять значение:

```
int x = 5;
int* ptr = &x;
*ptr = 6;
std::cout << x << std::endl; // 6
```

Использование звездочки `*`

- Символ используется в разных контекстах:

```
int x = 42;
// Умножение
auto mul = x * 42;
// Объявление указателя
int* ptr = &x;
// Разыменование указателя
```

```
auto y = *ptr; // copy
auto& ref = *ptr; // ref
```

Стиль кода амперсанда и звездочки

Это не обсуждается, используем первый вариант!

```
int* ptr = &x;
int& ref = x;
```

Так тоже можно:

```
int * ptr = &x;
int & ref = x;
```

Legacy языка C:

```
int *ptr = &x;
int &ref = x;
```

Может пригодиться при объявлении цепочки указателей

```
int *p = nullptr, *p1 = nullptr, *p2 = nullptr;
```

- **НО** нормальный *style guide* просит объявлять переменные в отдельной строке

Совместное использование амперсанда и звездочки

Операторы разыменования и взятия адреса можно использовать совместно

```
int x = 0;
int& ref = *&x; // взять адрес, а затем разыменовать
int* ptr = &x;
int* ptr2 = &*ptr; // разыменовать, а затем взять адрес переменной
```

Константность указателя

Указатель имеет две независимые константности:

- Константность данных - можно ли изменять объект через указатель. Квалификатор `const` стоит рядом с типом данных `const <type>*` или `<type> const*` (принято на практике использовать первый вариант)

- Константность хранимого адреса (самого указателя) - можно ли перевесить указатель на другой объект. Квалификатор `const` стоит после указателя `<type>* const`

Объявление удобно читать **справа налево**, от имени переменной:

- `const int* p` — `p` это указатель (`*`) на `int` , который `const` ;
- `int* const p` — `p` это константный (`const`) указатель (`*`) на `int` .

На константную переменную **НЕ** может указывать обычный указатель:

```
const int x = 5;
int* ptr = &x;  // compile error
int& ref = x;   // compile error
```

- аналогично для ссылок

На константную переменную может указывать указатель на константу:

```
const int x = 5;
const int* ptr = &x;
const int& ref = x;
*ptr = 18; // compile error
ref = 18; // compile error
```

- аналогично для ссылок

Константный указатель на константную переменную:

```
const int x = 5;
const int y = 18;
const int* const ptr = &x;
ptr = &y; // compile error
```

- нельзя менять (перевешивать на другую переменную) указатель и сами данные

На обычную переменную может указывать как константный указатель, так и указатель на константу.

```
int x = 5;
const int* ptr_const_data = &x;
int* const ptr_const = &x;
const int* const ptr_cc = &x;
```

- добавить константность можно, а вот убирать нельзя

Константный указатель, как и любая константа, обязан быть инициализирован при объявлении:

```
int* const p; // compile error: uninitialized 'const p'
```

Сводная таблица для `const` и указателя `*`:

Объявление	Изменить `*p`	Перевесить `p`
`int* p`	да	да
`const int* p`	нет	да
`int* const p`	да	нет
`const int* const p`	нет	нет

Нулевой указатель `nullptr`

`nullptr` - литерал специального типа `std::nullptr_t`, который неявно преобразуется в указатель любого типа. Введен в C++11 для обозначения нулевого указателя и используется для инициализации указателей, которые никуда не указывают.

Рассмотрим следующий пример:

```
double* ptr;
if (condition) {
    ptr = &value;
}
if (ptr) { // UB because reading uninitialised ptr
    std::cout << *ptr << std::endl; // UB because ptr has random address
}
```

Для правильной работы кода, необходимо инициализировать указатель значением `nullptr`:

```
double* ptr = nullptr;
```

До C++11 в качестве нулевого указателя использовался целочисленный тип, который записывался как `0` или макрос `NULL` из языка C. Поскольку `NULL` является целым числом (`0` или `0L`), а не указателем, то возникали проблемы с перегрузкой функций в такой ситуации:

```
void f(int)  { std::cout << "f(int)\n"; }
void f(int*) { std::cout << "f(int*)\n"; }

f(0);       // f(int)
f(NULL);    // compile error: call of overloaded 'f(NULL)' is ambiguous
f(nullptr); // f(int*) - nullptr всегда указатель
```

В современном C++ нулевой указатель обозначается **только** `nullptr`. `NULL` и `0` встречаются в *legacy* коде и коде на C.

Арифметика указателей

Над указателями можно проводить **арифметические операции** сложения и вычитания указателя с числом, инкремент/декремент.

```
int arr[5] = {1, 2, 3, 4, 5};
int* ptr = arr;   // pointer to 1
++ptr; // pointer to 2
ptr += 2; // pointer to 4
auto middle = ptr - 1; // pointer to 3
```

Арифметика указателей приводит к смещению указателя на другой блок памяти, поскольку указатель знает тип, на адрес объекта которого указывает, то известен и размер шага.

При арифметике указателей изменяется адрес, хранимый указателем на столько же байт, сколько отведено под хранение переменной типа.

Для неконстантных указателей, которые могут быть перевешены на другой объект, работают все вышеописанные операции.

Для константных указателей (`int* const`) возможны операции, **НЕ** изменяющие исходный указатель. Сложение и вычитание с числом возвращает новый указатель со смещенным значением.

Сравнение указателей

Сравнения `==` и `!=` применяются к любым указателям одного типа и к `nullptr`.

Сравнения `<`, `>`, `<=`, `>=` применяются к любым указателям одного типа.

Указатель на начало массива будет меньше указателя на конец.

Гарантированный результат сравнения обеспечивается только для указателей внутри одного массива (больше тот, у кого больше индекс). Для указателей на разные объекты результат не определен стандартом (*unspecified*), но это **НЕ** является UB.

Сравнение работает, в том числе, для константных указателей.

Расстояние между указателями

Можно определить расстояние между двумя указателями:

```
int arr[5] = {1, 2, 3, 4, 5};
int* begin = arr;   // pointer to 1
int* end = arr + 5; // pointer to pos after end of array
auto distance = end - begin;  // 5 type is std::ptrdiff_t
```

- `std::ptrdiff_t` - тип, используемый для расстояния между указателями

Связь массивов и указателей

В предыдущей лекции уже упоминалось встроенное в язык C++ неявное преобразование *array-to-pointer conversion* имени массива в указатель на первый элемент (сленговое именование *decay*).

```
int arr[5] = {10, 20, 30, 40, 50};
int* ptr = arr; // то же самое, что int* ptr = &arr[0];
```

При передаче в функцию массив превращается в указатель, поэтому размер внутри функции узнать нельзя:

```
void Size(int arr[]) {          // на самом деле int* arr
    std::cout << sizeof(arr);   // 8 - размер указателя!
}
```

Компилятор предупреждает об этом:

```
warning: ‘sizeof’ on array function parameter ‘arr’ will return size of ‘int*’
```

Оператор `[]` для встроенных массивов и указателей определен через арифметику указателей:

```
arr[i] == *(arr + i) // по определению
&arr[i] == arr + i   // адрес i-го элемента
ptr[i] == *(ptr + i) // для указателя работает точно так же
```

Поскольку сложение коммутативно, то `*(arr + 2) == *(2 + arr)`. Поэтому компилируется даже запись `2[arr]`. Так писать **НЕ** следует, это просто показывает, что оператор `[]` представляет собой сокращение для арифметики указателей.

Массив — это **НЕ** указатель. Различия проявляются в нескольких местах. Преобразование *array-to-pointer conversion* **НЕ** происходит в следующих случаях:

- При использовании оператора `sizeof(arr)`, возвращает размер всего массива в байтах.
- При взятии адреса `&arr`, возвращается указатель на весь массив `int(*)[5]`, а не только на первый элемент.
- При привязке к ссылке `auto& ref = arr`

```
int arr[5] = {10, 20, 30, 40, 50};
int* ptr = arr; // то же самое, что int* ptr = &arr[0];
auto ptr_arr = &arr; // int(*)[5]
auto& ref = arr; // int(&)[5]
```

```
std::cout << sizeof(arr) << std::endl;       // 20
std::cout << sizeof(ptr) << std::endl;       // 8
std::cout << sizeof(*ptr_arr) << std::endl;  // 20
std::cout << sizeof(ref) << std::endl;       // 20
```

Что это значит с практической точки зрения:

- адрес массива и адрес его первого элемента совпадают, но имеют **разные типы**, а значит и разный шаг арифметики: `++ptr` смещается на один элемент массива (4 байта), а `++ptr_arr` на весь массив (20 байт)
- указатель `ptr` можно перевесить на любой `int`, указатель `ptr_arr` можно перевесить только на массив `int` с таким же количеством элементов
- ссылка `ref` является корректным псевдонимом оригинального массива

Передача массива через указатели

Возможно передать массив в функцию через пару указателей.

Синтаксис функции: `<return_type> <func_name>(<type>* begin, <type>* end)`

Получение указателей на начало и конец:

```
int arr[5] = {1, 2, 3, 4, 5};
int* begin = arr;   // pointer to 1
int* end = arr + 5; // pointer to pos after end of array
```

В заголовочном файле `<iterator>` есть удобные функции для получения указателя на начало (`std::begin()`) и конец (`std::end()`) массива. Способ предпочтительнее в современном C++.

```
int arr[5] = {1, 2, 3, 4, 5};
int* begin = std::begin(arr);   // pointer to 1
int* end = std::end(arr);       // pointer to pos after end of array
```

Пример с функцией:

```
void Print(int* begin, int* end) {
    for (; begin != end; ++begin) {
        std::cout << *begin << std::endl;
    }
}

int main() {
    int arr[5] = {1, 2, 3, 4, 5};
    int* begin = std::begin(arr);   // pointer to 1
    int* end = std::end(arr);       // pointer to pos after end of array
    Print(begin, end);
```

```
    Print(std::begin(arr), std::end(arr));
}
```

Ссылка на указатель

Можно объявить ещё одно имя для указателя посредством ссылки:

```
int x = 0;
int* ptr = &x;
int*& ptr2 = ptr;
```

Указатель на указатель

Можно получить указатель на указатель. Нужно помнить, что каждая вложенность указателя увеличивает уровень косвенности (не прямого обращения), что может замедлять работу с исходным объектом под указателем.

```
int x;
int* ptr = &x;             // pointer to int
int** pptr = &ptr;         // pointer to pointer to int
int* const* npptr = &ptr; // non-const pointer to const pointer to int
```

Поскольку в языке есть преобразование `array to pointer`, то функцию `main` с аргументами можно записать двумя способами:

- `int main(int argc, char* argv[]) {<body>}`
- `int main(int argc, char** argv) {<body>}`

Указатель на `void`

Указатель на `void` — это указатель общего назначения, который может хранить адрес объекта любого типа, но не имеет информации о типе этого объекта.

Синтаксис: `void* <ptr_name>;`

Особенности:

- Может указывать на данные любого типа
- **НЕ** может быть разыменован напрямую
- **НЕ** поддерживает арифметику указателей
- Требует явного приведения для использования

Приведение перед использованием осуществляется с помощью `static_cast`. Приводить необходимо к тому же типу, какой был изначально. Приведение к другому типу **НЕ** проверяется компилятором и является **UB** (за исключением чтения байтов, о чем будет написано далее).

```
int number = 0x12345678;
int* ptr_int = &number;
void* ptr_void = ptr_int;
std::cout << std::hex;
// the same output
std::cout << "value int = " << number << '\n';
std::cout << "deref int* = " << *ptr_int << '\n';
std::cout << "deref (int*)void* = " << *static_cast<int*>(ptr_void) << '\n';
// the same output
std::cout << "address of int = " <<  &number << '\n';
std::cout << "address in ptr_int = " << ptr_int << '\n';
std::cout << "address in ptr_void = " << ptr_void << '\n';
// different address
std::cout << "address of ptr_int = " << &ptr_int << '\n';
std::cout << "address of ptr_void = " << &ptr_void << '\n';
```

Оператор `reinterpret_cast`

- `reinterpret_cast` - это оператор приведения, который интерпретирует битовое представление объекта одного типа как битовое представление объекта другого типа.

Синтаксис: `reinterpret_cast<new_type>(<expression>)`

- `new_type` - тип в который преобразовать
- `<expression>` - выражение, результат которого необходимо преобразовать

Не выполняет никаких преобразований.

Позволяет считать битовое представление числа в памяти:

```
int number = 0x12345678;
int* ptr_int = &number;
unsigned char* ptr_char = reinterpret_cast<unsigned char*>(ptr_int);

std::cout << "original int = " << std::hex << number << std::endl;
std::cout << "reinterpret as bytes = ";
for (size_t i = 0; i < sizeof(int); ++i) {
    std::cout << std::hex << static_cast<int>(ptr_char[i]) << " ";
} // 78 56 34 12
std::cout << std::endl;
```

- вместо `static_cast<int>(ptr_char[i])`, можно использовать унарный плюс для сокращения записи `+ptr_char[i]`, но `static_cast` более читабелен

Для чтения байтов лучше использовать указатель на `unsigned char`, а не `char`. Поскольку `char` может быть знаковым на некоторых платформах, то при чтении байта, имеющего значение больше или равного `0x80`, получится отрицательное значение. А при выводе в hex приведенного к `int` значения, получится большое число с ведущими единицами `FFFFFF`.

Можно провести эксперименты с типом `char` для различных архитектур процессоров, используя godbolt.org

---

Правило строгого алиасинга (*strict aliasing*)

Strict aliasing (строгий алиасинг) — это правило в C++, согласно которому объект, доступный через указатель/ссылку определенного типа, **НЕ** будет одновременно доступен через указатель/ссылку несовместимого типа.

Нарушение этого правила влечёт **UB**.

Совместимые (разрешенные) типы для обращения к объекту типа `T`:

- тип `T` с любыми cv-квалификаторами
- `signed` и `unsigned` вариант для интегрального `T`
- `char`, `unsigned char` и `std::byte` (*C++17*) для чтения байтов любого `T`

Данное правило позволяет компилятору производить оптимизации кода. Например, есть два указателя на разные типы, допустим `int*` и `float*`. В таком случае, компилятор вправе считать, что они указывают на разные объекты, и, следовательно, **НЕ** перечитывать значение из памяти после записи через другой указатель.

```
int Change(int* i, float* f) {
    *i = 1;
    *f = 0.0f;   // компилятор считает, что float* не может указывать на int
    return *i;   // поэтому при -O2 подставляет 1, не перечитывая память
}

int main() {
    int x = 0;
    int result = Change(&x, reinterpret_cast<float*>(&x)); // UB: strict
aliasing
    std::cout << "result = " << result << ", x = " << x << '\n';
}
```

- без оптимизаций (флаг `-O0`) значение перечитывается из памяти и получается `0`;
- с оптимизацией (флаг `-O2`) функция возвращает `1`, хотя в `x` лежит `0`.

Чтение переменной `float` как `int`, тоже является **UB**

```
float f = 1.0f;
int* ptr = reinterpret_cast<int*>(&f);
int bits = *ptr; // UB: объект float читается как int
```

Интерпретировать байты одного типа как другой тип, можно безопасно следующими способами:

1. Использовать `std::memcpy` из `<cstring>`

```
float value = 1.0f;
unsigned int bits = 0;
std::memcpy(&bits, &value, sizeof(value));
```

- компиляторы оптимизируют `std::memcpy` небольшого размера в простое копирование регистра, накладных расходов нет;

2. Использовать `reinterpret_cast` к `unsigned char`, `char`, `std::byte`

```
float value = 1.0f;
auto* bytes = reinterpret_cast<unsigned char*>(&value);
```

- после чего можно пройтись по `bytes`, как по массиву, от `0` до `sizeof(float)`

3. Использовать `std::bit_cast` (C++20) из `<bit>`

```
float value = 1.0f;
auto bits = std::bit_cast<unsigned int>(value);
```

- `std::bit_cast` требует, чтобы типы имели одинаковый размер, это проверяется на этапе компиляции.

GCC и Clang включают правило *strict aliasing* на оптимизациях:

- `-fstrict-aliasing` — включено, например, на `-O2`
- `-fno-strict-aliasing` — отключает оптимизации на основе данного правила

С флагом `-fno-strict-aliasing` собирается, например, ядро Linux, но это **НЕ** делает код корректным с точки зрения **Стандарта**.

В MSVC нет такого флага, поскольку используется своя модель алиасинга. В других компиляторах опция может называться по-другому. Реализация работы опции может отличаться от версии компилятора.

Правильно писать код, **НЕ** нарушающий правила, а **НЕ** отключать посредством флага компиляции, так как это ухудшает оптимизации и **НЕ** портируемо

GCC может предупреждать о некоторых нарушениях правила с флагами `-O2 -Wall` (`-Wstrict-aliasing`):

```
warning: dereferencing type-punned pointer might break strict-aliasing rules
```

Предупреждение срабатывает далеко **НЕ** всегда, поэтому полагаться на него нельзя.

Статический анализ нарушений *strict aliasing* на уровне компилятора может давать слишком много ложных срабатываний и **НЕ** покрывает все случаи.

В Clang 20+ реализован экспериментальный инструмент динамической диагностики TypeSanitizer (`-fsanitize=type`). При этом компилятор вставляет дополнительный код и производит проверки нарушения *strict aliasing* **во время выполнения** программы. Естественно, такой инструмент используется только в отладочных сборках, **НЕ** в релизных, поскольку имеет большие накладные расходы.

Указатель на функцию

Указатель на функцию — это переменная, которая хранит адрес функции, а не данных.

Позволяет:

- Передавать функции как аргументы в другие функции
- Создавать массивы функций
- Реализовывать стратегии и обратный вызов функции (*callback*)

Синтаксис объявления указателя на функцию: `<return_type> (*<ptr_name>)(<arg_types>);`

- `<return_type>` - возвращаемое значение функции
- `<ptr_name>` - имя указателя
- `<arg_types>` - типы аргументов функции

Преобразование *function to function pointer*

В языке присутствует стандартное преобразование функции в указатель на функцию. Поэтому там, где требуется указатель на функцию, **НЕ** обязательно передавать адрес функции, можно просто передать имя функции.

```
int Add(int a, int b) {
    return a + b;
}

int Mul(int a, int b) {
    return a * b;
}

int Apply(int (*op)(int, int), int x, int y) {
    return op(x, y);
}

std::cout << Apply(Add, 2, 3) << std::endl;   // 5
std::cout << Apply(&Mul, 2, 3) << std::endl;  // 6
```

Сложные объявления с указателями

Ниже приведены примеры сложных объявлений с участием указателей:

```
int* arr[10];                   // array of 10 int* elements
int (*ptr_arr)[10];             // pointer to array with 10 int elements
void (*ptr_func)(int);          // pointer to function with type void(int)
void (*arr_ptr_func[10])(int);  // array of 10 pointers to functions of type
void(int)
void (*(*arr_ptr_func_r_ptr_func[10])(int))(int);
// array of 10 pointers to functions that take an int and return a pointer to a
function of type void(int)
```

Передача аргументов в функцию

Передача по значению, копируем объект:

```
int func(std::string s);
```

Передача по ссылке, не копируем объект, можем изменять существующий:

```
int func(std::string& s);
```

Передача по константной ссылке, не копируем объект, нельзя изменять существующий:

```
int func(const std::string& s);
```

Передача по указателю, не копируем объект, можем изменять существующий:

```
int func(std::string* s);
```

Передача по указателю на константу, не копируем объект, нельзя изменять существующий:

```
int func(const std::string* s);
```

Продление жизни временного объекта ссылкой на константу

Синтаксис: `const <type>& ref = <entity>`

- Константная ссылка на временный объект создает себе объект и становится его именем

Константные ссылки можно инициализировать через ***rvalue*** (*lifetime extension*)

Почему нет продления жизни неконстантной ссылки

Если бы продление жизни неконстантной ссылки выполнялось, то следующий пример скомпилировался бы:

```
void func(size_t& counter) {
    ++counter;
}

int main() {
    int x = 0;
    func(x);
    std::cout << x << std::endl;
}
```

Но это явно **НЕ** то, что хотел бы пользователь. Если бы это работало, то создавался бы временный объект `counter` из `x` посредством приведения типа `int` к `size_t` и инкремент бы происходил над временным объектом. В данном случае, конечно, лучше получить ошибку компиляции, что `int` и `size_t` не совпадают.

Висячая ссылка (*dangling reference*)

В C++ не запрещено возвращать ссылку на локальную переменную, но поскольку по выходу из функции локальная переменная уничтожается, то ссылка становится висячей (битой) и обращение к ней приведет к **UB**, необратимым последствиям:)

```
int& GetLocal() {
    int x = 5;
    return x;
}

int main() {
    int& x = GetLocal();
    std::cout << x << std::endl; // UB
}
```

Компилятор может выдавать предупреждение, это зависит от версии.

```
GCC: warning: reference to local variable ‘x’ returned [-Wreturn-local-addr]
Clang: warning: reference to stack memory associated with local variable 'x'
returned [-Wreturn-stack-address]
```

Невалидный указатель (*invalid pointer*)

Нельзя разыменовывать указатель, который указывает на память, уже не принадлежащую объекту. Шучу, это же C++: разыменовать можно, но программа скорее всего упадет, так как разыменование невалидного указателя — это **UB**!

```
int main() {
    int* ptr = nullptr;
    {
        int x = 42;
        ptr = &x;
    }
    std::cout << *ptr << std::endl; // UB
}
```